광주 졸업식 일정 도우미
프로젝트 결과 보고서

G118 김희주

Chapter 1. 진행 사항과 프로젝트 소개

1. 선정 주제와 개발 배경

과제 2: 신규 AI 서비스 개발로 광주 지역 초·중·고·대학의 졸업식 날짜, 실제 시작 시각, 관련 인원을 조사하는 로컬 웹 프로그램을 만들었습니다. 제공된 샘플 ZIP 코드를 직접 재사용하지 않고 새로 구현했습니다.

졸업식 꽃다발을 준비하는 사용자는 여러 학교의 일정과 위치를 비교해야 합니다. 그러나 정보가 학사일정·공지·첨부 문서에 흩어져 있고, 날짜만 공개되거나 식전 행사와 본식 시각이 함께 적혀 있어 검색 결과만으로 정확한 시간을 판단하기 어렵습니다.

목표는 근거가 있는 정보만 제공하고, 부족한 값은 직접 보완할 수 있는 일정표입니다. 사용자는 기간을 지정해 조사하고 직접 확인한 내용을 기록하며, HTML 결과는 프로그램 없이 조회합니다. 판매 가능 여부나 수요는 사용자가 판단합니다.

2. 주요 Task와 완료 범위

표 1. 주요 Task와 구현 결과

주요 Task	구현 결과
학교 목록·기간별 조사	공식 명부를 불러오고 실제 행사일 범위를 시즌으로 저장합니다. 기존 시즌의 재조사와 새로운 기간의 조사를 구분합니다.
날짜·인원·시간 수집	기본 조사는 전체 대상 학교의 날짜·인원을 찾습니다. 같은 원문에 본식 시각이 있으면 저장하고, 별도 시간 검색은 학교별 버튼으로 실행합니다.
근거와 관계 검증	기관·행사·학년도·원문 인용을 검사합니다. 시각만 불확실하면 날짜는 유지하며, 취소·미실시와 부대 행사 시각을 구분합니다.
직접 확인 내용 기록	자연어 입력을 기존 행사에 연결합니다. 모호한 항목은 질문하고, 원문·실제 확인시점·정규값·수정 이력을 보존합니다.
간단한 결과 제공	날짜·학교·시간·인원·학교 주소의 표를 제공합니다. 검색·필터·20행 페이지 이동·전체 조건 인쇄가 가능한 독립 HTML을 생성합니다.
지속 저장과 복구	SQLite에 기록을 보존하고 일시정지·재개·취소·백업·기록 초기화를 지원합니다. 운영 자료와 검증용 자료를 분리했습니다.

구현 기준: 저장소 gwangju-graduation-search-agent, 커밋 `b7f94b5`. 보고서의 "완료"는 기능 구현·검증 범위이며, 모든 학교의 모든 정보가 수집되었다는 뜻은 아닙니다.

Chapter 2. 사용 기술과 구현 결과

1. 개발 환경과 실행 방법

표 2. 확인한 환경과 기술 구성

구분	적용 내용
실행 환경	macOS 26.5 · arm64 · Python 3.11.15. 프로젝트 지원 범위는 Python 3.11~3.13입니다.
웹·저장	FastAPI 0.151.0, Uvicorn 0.54.0, HTML/CSS/JavaScript, 로컬 SQLite.
연동·검증	HTTPX 0.28.1, MCP SDK 1.30.0, Pydantic·JSON Schema, HTML·텍스트 PDF 읽기.
모델	Ollama/OpenAI 중 설정한 하나를 사용합니다. 본 보고서의 실제 수집 검증은 **Ollama qwen3:8b**로 수행한 기록입니다. OpenAI 실호출 성공을 이번 결과에 포함하지 않습니다.
자료원·관찰	NEIS 학사일정, 학교알리미 학생 수, Tavily 검색, 공식 학교 문서. LangSmith 0.14.11은 선택적 추적에 사용합니다.

프로젝트 폴더에서 다음 명령으로 환경을 준비합니다. `.env.example`을 참고해 `.env`를 만들고 자료원 키와 선택한 모델 연결을 설정한 뒤 `start.command`를 실행하면 브라우저에서 앱이 열립니다.

```
python3 -m venv .venv
.venv/bin/python -m pip install -r requirements.lock
.venv/bin/python -m pip install --no-deps -e .
./start.command
```

사용 흐름은 **학교 목록 불러오기 → 새 시즌 조사 → 필요한 시간 찾기·직접 기록 → 파일 내려받기**입니다. 기본 DB는 `data/live/graduation.sqlite3`, 자동 저장 HTML은 `data/live/exports/`에 보관합니다. 종료는 화면의 **프로그램 종료**를 사용합니다.

출처: 프로젝트 `README.md`, `pyproject.toml`, `app/launcher.py`, 보고서 작성 시점의 설치 패키지 조회. 비밀값과 사용자 기록은 보고서에 포함하지 않았습니다.

2. 시스템 구조와 Agent의 역할

학교별 조사는 하나의 `ResearchRunner`가 담당합니다. 먼저 공식 정형 자료와 검증된 학교 경로를 확인하고, 부족한 정보가 남으면 모델이 **검색·원문 읽기·종료** 중 다음 행동을 선택합니다. 실행 결과와 실패 이유는 다음 판단의 문맥으로 돌아갑니다.

모델의 선택을 곧바로 사실로 저장하지 않습니다. 코드는 도구 권한·중복 요청·예산을 검사한 뒤 행동을 실행하고, 후보 값은 별도의 기관·행사·근거 검사로 승인합니다. 정형 자료로 충분한 경우에는 모델을 호출하지 않습니다.

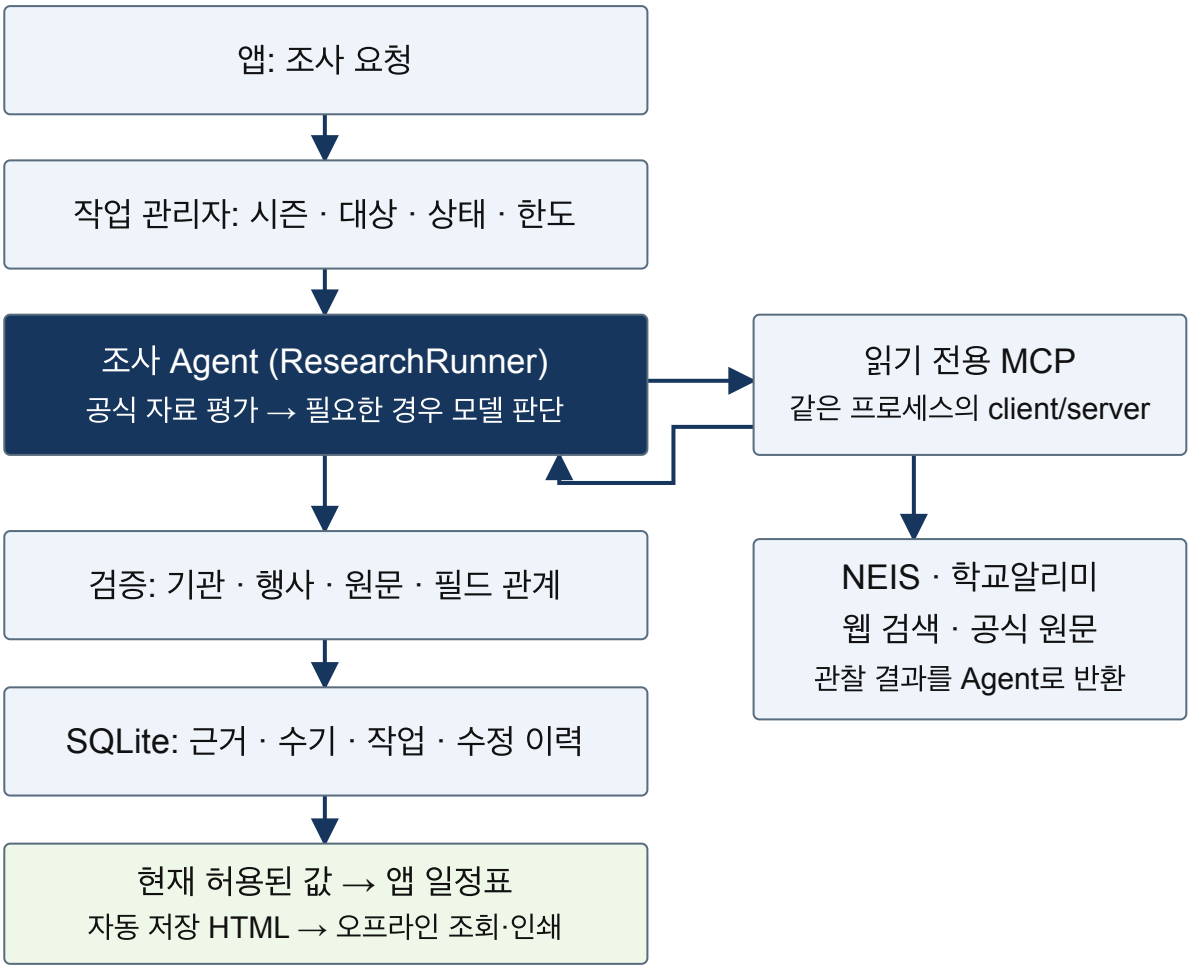


그림 1. 조사 처리 구조를 설명한 개념도. 직접 기록은 별도의 로컬 파서·검증을 거쳐 SQLite에 저장됩니다.

시즌은 양끝을 포함하는 실제 행사 날짜 범위이며 달력상 6개월 미만입니다. 자료의 학년도는 행사 원문에서 확인합니다. 예를 들어 2026년 2월 행사가 2025학년도 행사일 수 있으므로 날짜의 연도 하나로 학년도와 인원을 정하지 않습니다. 동일 행사의 정본과 시즌 연결을 분리해 겹치는 시즌에서도 수기·근거를 중복 복제하지 않습니다.

3. 실제 적용한 기술 요소와 선택 이유

표 3. 기술 적용 범위와 구현상의 선택

기술	실제 적용	선택 이유·제약
Agent 실행 반복	현재 근거·누락·실패·남은 예산을 보고 다음 행동을 정하고, 관찰 후 다시 판단합니다.	정해진 검색 순서보다 상황에 맞게 탐색합니다. 무한 반복을 막기 위한 한도가 필요합니다.
MCP	학사일정·검색·원문 읽기·학년 인원 도구 4개와 검증 정책 Resource 1개를 실제 SDK 세션으로 호출합니다.	도구의 입력·출력과 기관·기간을 제한합니다. 메모리 연결이며 독립 네트워크 서버는 아닙니다.

기술	실제 적용	선택 이유·제약
의존성 주입	생성자 인자로 HTTP client, 조사 실행기, 추적 client 등을 전달합니다.	검사에서 외부 장애·응답을 재현합니다. 별도 DI 컨테이너는 사용하지 않습니다.
구조화 응답·검증	행동·후보를 JSON으로 받고 스키마와 원문 인용·관계를 다시 검사합니다.	형식의 유효성과 사실의 정확성을 분리합니다. JSON이 정상이어도 근거가 없으면 승인하지 않습니다.
LangSmith	작업 → 학교 → 모델 결정·도구 호출의 관계와 허용한 상태·오류·사용량을 선택적으로 추적합니다.	문제 지점을 찾되 원문·수기·키는 전송하지 않습니다. 전송 실패는 조사와 분리합니다.
SQLite·이력	시즌·행사·근거·직접 기록·작업 상태를 거래 단위로 저장하고 백업·재시작을 지원합니다.	별도 DB 서버 없이 운영합니다. 다중 사용자 서비스는 범위에 넣지 않았습니다.
모듈화	`core`, `agent`, `integrations`, `validation`, `records`, `web`, `static`으로 책임을 구분했습니다.	찾아 수정할 위치를 명확히 합니다. 큰 모듈의 내부 분할은 일부 남아 있습니다.
단위·통합·UI 검사	pytest 회귀 검사, 실제 실행 수명 검사, 전용 브라우저 검사, 별도 실제 외부 수집 검증을 수행했습니다.	합성 입력의 성공을 실제 수집 정확도로 환산하지 않습니다.

적용하지 않은 요소: Multi-Agent 협업, 임베딩·벡터 DB 기반 RAG, LangGraph, Vue, MCP Prompts는 사용하지 않았습니다. 단일 조사 Agent와 코드 검증으로 현재 요구를 처리하고, 관찰 가능한 복잡도를 유지하는 편이 적절하다고 판단했습니다. 과거 공식 경로의 재사용은 모델 학습이나 벡터 검색을 뜻하지 않습니다.

자연어 직접 기록은 **로컬 규칙 파서**로 처리합니다. 전화 확인 메모를 외부 LLM에 보내지 않고 지원하는 표현을 정규화하며, 실제 확인시점이 빠졌거나 기존 정보와 충돌하면 질문합니다. 파싱에 성공했다는 이유로 사실이 외부에서 검증되었다고 표시하지 않습니다.

코드 근거: `app/agent/research.py`, `tool_boundary.py`, `tracing.py`, `app/records/natural.py`, `app/validation/`. MCP와 추적 개념은 MCP 공식 문서, LangSmith 공식 문서를 참고했습니다. 표는 해당 개념 전체가 아닌 이 프로젝트의 구현 범위입니다.

4. 주요 기능별 실행 화면

다음 기능 화면은 현재 코드를 실제 브라우저에서 실행해 캡처했습니다. 학교·날짜·전화 확인 진술이 들어간 기능 검사는 **운영 데이터와 분리한 합성 자료**를 사용했습니다. 실제 외부 수집 결과 화면은 뒤의 5절에 따로 제시합니다.

4.1 학교 목록과 기간 기반 시즌

학교 목록이 없으면 불러오기 동선을 안내합니다. 새 시즌은 행사일 범위를 선택해 만들며, 같은 기간을 다시 조사할 때 기존 시즌과 기록을 유지합니다. 날짜 역전, 말일·윤년과 6개월 경계는 화면과 서버에서 검사합니다.



그림 2. 학교 목록이 없는 최초 실행 화면. 새 격리 DB이며 조사 결과는 없습니다.

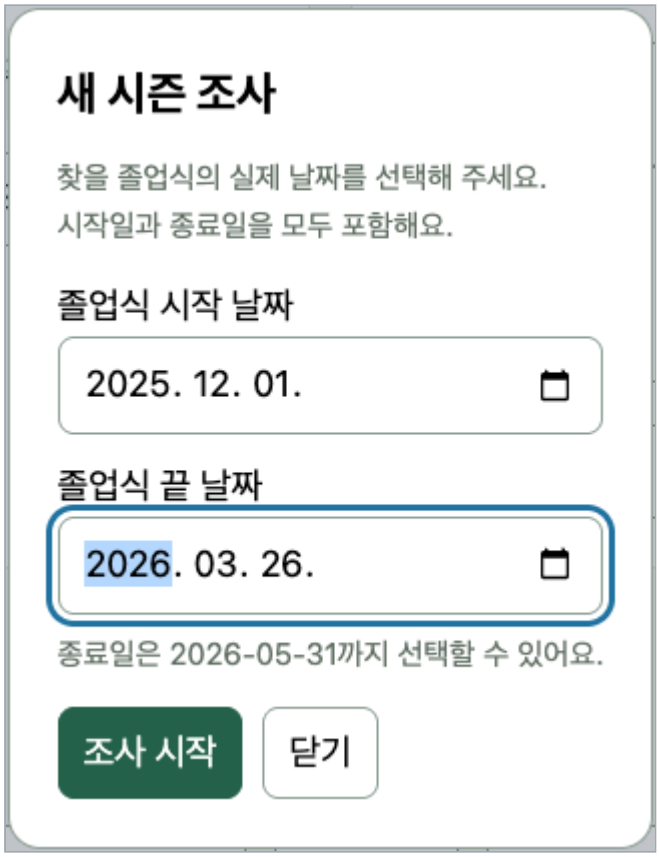


그림 3. 실제 행사일 범위를 지정하는 새 시즌 입력. 화면 검증용이며 이 입력으로 외부 조사를 시작하지 않았습니다.

4.2 학교별 시간 찾기와 작업 상태

시간 찾기는 해당 시즌·행사 ID에 연결됩니다. HTTP 응답 완료가 아니라 실제 작업 상태를 표시하며, 새로고침 후에도 복원합니다. 다른 행의 버튼이 비활성화되더라도 그 행을 조사 중이라고 표시하지 않습니다.



그림 4. 해당 학교의 시간 칸에만 표시되는 진행 상태. 합성 자료·제어된 조사 실행기를 사용했습니다.

미발견, 자료 접근 실패, 일시정지, 취소는 같은 위치에서 구분합니다. 취소해도 이미 저장된 근거는 남습니다. 인쇄에서는 조사 버튼과 진행 문구를 숨깁니다.

4.3 자연어 직접 기록과 기존 행사 보완

아래 입력은 기능 검사를 위한 가상의 전화 확인 진술입니다. 기존 행사에 오전 9시를 보완한 뒤 **행사 ID 목록과 건수 43개가 그대로 유지**되어 새 행사가 생기지 않은 것을 확인했습니다.



그림 5. 자연어 직접 기록 입력. 가상의 학교·전화 확인 진술로 수행한 기능 검사입니다.

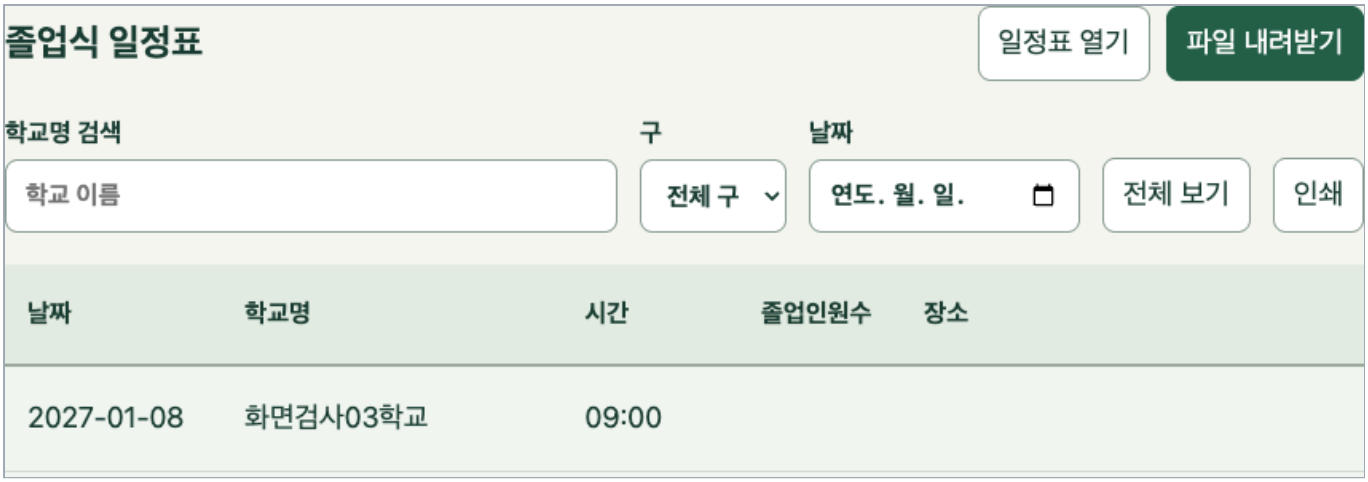


그림 6. 기존 합성 행사의 시간을 09:00으로 보완한 결과. 행사 ID와 건수는 유지됐습니다.

4.4 조건 조회·페이지 이동·인쇄

학교명·구·날짜 조건을 함께 적용하고 20행씩 표시합니다. 날짜순 필터 결과 전체에 그룹 색을 배정한 다음 페이지를 나누므로 같은 날짜가 페이지 경계에 걸려도 같은 색입니다. 인쇄는 현재 페이지가 아닌 조건에 맞는 전체 행을 대상으로 합니다.



그림 7. 합성 45개 학교 중 이름·날짜 조건에 맞는 4행을 조회한 화면.

기간별 HTML은 자동 갱신됩니다. 내려받은 파일에는 CSS와 JavaScript가 포함되어 조회에 서버가 필요하지 않습니다. 다만 내려받은 사본은 이후 조사 결과를 자동 반영하지 않으며, 학교 홈페이지 방문에는 인터넷이 필요합니다.

5. 실제 수집 결과와 최종 실행 화면

2026-10-22 10:58~11:02의 별도 실제 검증에서는 공식 학교 프로필만 준비하고 **일정·시각·인원 정답을 미리 넣지 않았습니다.** Ollama qwen3:8b와 외부 자료원을 사용한 결과입니다. 아래 화면은 그 검증 DB를 일관성 있게 복사해 현재 앱에서 다시 연 것이며, 캡처 과정에서 새 외부 조사를 수행하지 않았습니다.

표 4. 실제 API·모델 조사 결과와 미확보 항목

대상·기간	확인한 결과	미확보·실행 한계
GIST 2025.12.01~ 2026.03.26	**2026-02-20 14:00** 원문의 학년도: 2025	인원 미확보. 추가 원문 1건 접근 실패와 판단 횟수 한도 종료가 남았습니다.
건국초 2026.12.01~ 2027.03.31	**2027-01-06 · 188명** 2026학년도 6학년 재학생 수	시간 미확보. 기본 날짜·인원 조사는 정형 근거로 충분해 모델 호출 없이 완료했습니다.
건국초 과거 조사· 미래 시간 추가 조사	미래의 기존 날짜·인원은 유지	과거 날짜·시간·인원과 미래 시작 시각은 확보하지 못했습니다. 미공개·미실시 판정은 하지 않았습니다.

근거: `runtime/agent-recovery-20261022/run-105859/completion.json` 및 `report.json`. GIST 날짜·시각의 공개 근거는 2025학년도 학위수여식 공식 안내입니다. 건국초 인원은 학교알리미 API와 공식 공개 화면의 해당 학년도 자료로 확인했습니다.



그림 8. 실제 수집 결과의 날짜·인원과 미확보 시각. 188명은 2026학년도 건국초 6학년 재학생 수입니다.



그림 9. 실제 수집 결과 사본에서 생성한 정적 HTML 일정표. 앱의 조사·입력 기능은 포함하지 않습니다.

한계: GIST 공지에는 학위수여대상자 수가 있지만 해당 자동 조사에서는 인원을 검증·저장하지 못했습니다. 원문에 없는 정보와 프로그램이 찾아내지 못한 정보를 구분해야 합니다. 현재 표의 장소는 학교 주소이고, 초중고 인원은 졸업 학년 재학생 수이므로 실제 행사장·참석 인원을 확정하지 않습니다.

2026-10-21의 별도 API 검증에서는 첨단고의 2026-01-21 행사에 2025학년도 고3 355명, 2027-01-20 행사에 2026학년도 고3 305명을 연결했습니다. 행사 연도와 학년도를 분리하고 학교알리미 요청에 필수 시군구 코드를 포함하는 경로를 확인한 사례입니다. 당시 명부 검증의 기본 대상은 339개였으나, 이 수를 전체 일정 확보 건수로 사용하지 않았습니다.

6. 검증 결과와 실행 관찰

표 5. 검사 유형별 결과와 해석 범위

검사 유형	결과	검사 내용·제한
pytest 회귀	**968개 통과**	실패·오류·생략 0. 단위·통합·정확성 반례를 포함한 전체 묶음입니다.
실제 실행 수명	**7개 통과**	start.command, 중복 실행, 포트 충돌, 기록·백업·재시작·종료. DB와 기록은 검증용입니다.
실제 브라우저 UI	**101개 확인**	진행·복원·직접 보완·시즌·필터·색상·페이지·인쇄·다운로드. 외부 수집 정확성 검사는 아닙니다.

검사 유형	결과	검사 내용·제한
기존 자료 보존	219개 파일 동일	DB 사본의 시즌·기록 수와 무결성 검사. 사용자 데이터·설정 파일의 전후 해시가 일치했습니다.
실제 API·모델	일부 일정 확보	표 4의 실제 결과입니다. 전체 학교의 정확도·수집률이나 모든 값 확보를 입증하지 않습니다.
LangSmith 수신	실수신 확인 일부 누락 있음	작업→학교→모델/도구 관계와 실행 ID 수신을 확인했습니다. 최신 과거 조사에서는 전송 31개·누락 3개가 기록되었습니다.

회귀·실행·UI 검사는 2026-10-22 구조 변경 커밋 `f9fab45`에서 수행했고, 이후 `b7f94b5`는 README만 수정했습니다. 이 보고서에서는 기능 화면과 실제 검증 DB 사본을 다시 캡처했습니다. 서로 다른 날짜의 검사 개수를 더해 성과를 부풀리지 않았습니다.

식전 공연·촬영·준비 설명회 시각, 병설유치원 등 다른 기관의 행사, 별도 URL의 취소 공지, 미실시 부정 문맥, NEIS/HTML의 같은 행사 날짜 충돌, 직접 입력 중복, 동시 재개 제한을 반례로 검사했습니다. 정상 사례도 함께 유지해 모든 결과를 보류하는 방식으로 통과시키지 않았습니다.

검증 제한: 전용 브라우저 도구가 `file:` URL을 차단해 로컬 파일 경로를 직접 여는 동작은 확인하지 못했습니다. 대신 서버 종료 후 실제 다운로드한 HTML과 같은 바이트를 `data:` URL로 열어 검색·필터·페이지 이동·인쇄를 확인했습니다. 새 환경 전체 재설치, OpenAI 실호출, 전체 학교 전수 조사는 이번 최종 검증 완료 범위에 포함하지 않습니다.

LangSmith에는 허용된 실행 메타데이터만 보내며, 로컬 DB를 기본 이력으로 유지합니다. 최신 검증의 일부 전송 실패는 조사 결과 저장을 막지 않았습니다. 토큰 표시는 응답에서 확인한 값이며 전체 재시도 비용이나 청구액으로 해석하지 않습니다.



그림 10. 최종 앱 실행 화면. 2026-10-22 실제 수집 검증 DB 사본을 현재 앱으로 조회했습니다.

Chapter 3. 문제 해결 경험과 개선 방향

기간별 조사, 직접 확인 보완, 지속 저장, 독립 HTML 제공을 구현했고 일부 실제 일정의 확보를 검증했습니다. **모든 정보를 자동으로 얻는 완전성은 확보하지 못했습니다.** 확인하지 못한 값을 숨기거나 추정하기보다, 원인과 근거를 남기고 직접 보완할 수 있도록 구성했습니다.

1. 구현 과정의 어려움과 해결

표 6. 주요 문제와 조치·재검증

증상·원인	조치	재검증과 남은 한계
관련 없는 시각을 본식 시각으로 혼동	키워드만 찾는 방식 대신 기관·행사 주어·날짜·시각 역할·원문 구간을 연결했습니다.	부대 행사·다른 기관·부정·취소 반례를 검사했습니다. 모든 문서 표현을 처리하지는 못합니다.
시즌과 학년도를 하나로 취급	시즌은 행사일 범위, 학년도는 원문 사실로 분리하고 인원을 행사 학년도에 연결했습니다.	연도·학년도 경계와 연기 행사 회귀 검사, 첨단고 과거·미래 실제 자료 연결을 확인했습니다.
재조사·직접 기록의 중복	행사 정본과 시즌 연결을 구분하고, 직접 확인값은 보호하며 같은 행사에 누락값을 보완합니다.	합성 UI 입력 후 행사 43개의 ID가 유지됐습니다. 모호한 동일 행사 관계는 질문·보류합니다.
모델의 반복 행동·응답 형식 오류	가능한 행동만 제시하고 구체적인 오류를 돌려줍니다. 형식 수정·반복 복구 기회는 제한합니다.	최신 실제 검증에서 최종 형식 오류 0건. 시간 찾기에서는 반복 제안 2회 차단·복구 1회 후에도 미확보가 남았습니다.
내부 한도 소진을 외부 API 제한과 혼동	한도 종류·예약량·사용량을 기록하고, 호출 전 예약과 응답 후 정산을 유지했습니다.	기본 학교별 한도는 48,000토큰·모델 8회·판단 6회·180초입니다. GIST는 판단 한도로 종료했으며, 종료 시점까지 인원을 확보하지 못했습니다.

학교 사이트의 일부 잘못된 응답 헤더도 읽기 실패의 원인이었습니다. 확인된 특정 헤더 오류에 한해 제한적인 읽기 호환 경로를 두고, TLS 검증·공개 주소 제한·응답 크기·시간 한도를 유지했습니다. 단순한 경로 꼬임이나 시간 미공개로 모든 실패를 설명하지 않도록 했습니다.

2. 느낀 점

이번 프로젝트에서 중요하게 본 것은 기술의 개수보다 **실제 사용자가 끝까지 사용할 수 있는 결과**였습니다. 조사를 수행하는 Agent가 있어도 화면이 복잡하거나 결과를 프로그램 없이 열 수 없다면 사용 목적을 충분히 해결하지 못합니다. 조사·입력은 앱에 두고, 조회·인쇄는 독립 HTML에 두는 구성이 요구에 맞았습니다.

또한 모델 응답 형식이 정상인 것과 정보가 정확한 것은 서로 다른 문제였습니다. 시간 하나를 저장할 때도 학교, 본식과 부대 행사, 학년도, 취소 여부를 함께 확인해야 했습니다. AI는 탐색을 맡고 코드가 승인 기준을 지키도록 역할을 나눈 것이 핵심 설계 경험이었습니다.

검증에서도 정상 화면을 만드는 것만으로는 충분하지 않았습니다. 합성 자료는 중복·충돌·중단 같은 상황을 재현하는 데 유용하지만, 실제 수집 성공을 대신하지 못합니다. 실제 외부 호출의 성공과 실패를 따로 남기는 것이 결과의 신뢰성을 설명하는 데 중요하다는 점을 배웠습니다.

3. 향후 개선 사항

1. **미확보 원인의 우선순위에 맞춘 수집 개선:** 실제 실패 문서 중 빈도가 높은 형식부터 대응하고, 특히 원문에 있는 인원·시각을 놓치는 사례를 대표 평가 자료로 관리합니다. OCR·HWP 처리는 필요성과 검증 비용을 확인한 뒤 추가합니다.
2. **근거 확인 동선 개선:** 현재 학교명 링크는 공식 홈페이지입니다. 사용자가 필요할 때 해당 값의 공식 안내 원문으로 바로 이동할 수 있는 간단한 동선을 검토합니다.
3. **실사용 검증 보완:** 사용자 전체 사용 흐름과 더 넓은 기관 표본을 확인하고, 날짜·시각·인원을 나눈 확보율과 오류 유형을 기록합니다. 일부 성공을 전체 정확도로 확대하지 않습니다.
4. **운영 품질 보완:** LangSmith 전송 누락의 관찰 가능성을 높이고, 모델 변경 시 기존 반례를 다시 검증합니다. Multi-Agent나 벡터 DB는 현재 문제 해결에 필요한 근거가 생길 때 검토합니다.

참고자료와 검증 근거

과제 기준: 공지사항.md, 「AI 서비스 Mini-Project.pdf」와 결과 보고서 필수 작성 항목. 아래 실행 원문·검증 DB·설정은 공개 저장소에서 제외하며, 보고서에는 검토한 요약과 화면을 담았습니다. 캡처는 전용 브라우저에서 수행했고 운영 DB는 수정하지 않았습니다.

근거	프로젝트 안의 위치
현재 구현·사용	`README.md`, `docs/ARCHITECTURE.md`, `docs/RESULT_CONTRACT.md`, `checks/README.md`
회귀·UI·보존	`runtime/refactor-20261022/summary.json`
실제 수집	`runtime/agent-recovery-20261022/run-105859/completion.json`, 같은 폴더의 `report.json`
학년도·인원	`runtime/final-validation-20261021/cheomdan-past/report.json`, `cheomdan-future/report.json`, `runtime/sources-20261021/verification-summary.json`
추적 실제 수신	`runtime/minimal-finish-20261022/future/time-report.json`, 최신 실제 수집 `report.json`
보고서 화면·감사	`runtime/report-20261022/capture-summary.json`, `technical-audit.md`, `verification-audit.md`